

INSTITUTO TECNOLÓGICO SUPERIOR DEL OCCIDENTE DEL ESTADO DE HIDALGO

DIVISIÓN DE INGENIERÍA EN SISTEMAS COMPUTACIONALES

Manual Técnico de Firmware y Código Fuente Embebido

Pulsera Médica de Arteria Radial para Monitoreo Hemodinámico Continuo

Microcontrolador ESP32 Mini, Sensado Biofotónico MAX30102, Acelerometría MPU6050, Servidor BLE GATT, Monitoreo Autónomo de Batería con Histéresis y Carcasa CAD 2.0

Proyecto:

Monitor Continuo No Invasivo de Signos Vitales (BPM, Presión Arterial PWA y SpO2) con Cancelación Adaptativa de Ruido de Movimiento y Compensación Melánica Cutánea

Asesores:

- Mtra. Lorena Mendoza Guzmán
- Mtra. Cristy Elizabeth Aguilar Ojeda

Alumno / Desarrollador:

- Alejandro Pérez

Grupo:

- 8.º Semestre - Grupo A
-

Contenido

- **1. Arquitectura de Firmware y Definiciones Globales**
 - 1.1 Librerías del Sistema, Mapeo de Puertos y Declaración de UUIDs GATT
 - 1.2 Búferes de Acumulación y Estructuras de Datos de Telemetría
 - **2. Subsistema de Sensores sobre Bus I2C Compartido (400 kHz)**
 - 2.1 Inicialización de Hardware e Interfaz I2C Rápida
 - 2.2 Parametrización Óptica y Registros Biofísicos del Sensor MAX30102
 - 2.3 Configuración de Registros y Adquisición Triaxial del Acelerómetro MPU6050
 - **3. Subsistema de Energía y Gestión de Batería**
 - 3.1 Conversión Analógica-Digital en ADC1 (GPIO 34) y Divisor Resistivo
 - 3.2 Máquina de Estados de Histéresis Clínica de Seguridad (3.40V / 3.50V)
 - **4. Subsistema de Comunicación Inalámbrica BLE GATT**
 - 4.1 Configuración del Servidor GATT y Servicio Diagnóstico
 - 4.2 Empaquetamiento y Notificación de la Trama Extendida de 20 Bytes (50 Hz)
 - 4.3 Característica de Telemetría de Batería y Banderas de Estado (UUID 0x2A19)
 - 4.4 Receptor Asíncrono de Comandos de Control (START / STOP)
 - 4.5 Callback de Reconexión Automática Reactiva ante Interrupción de Enlace
 - **5. Bucle Principal Concurrente y Temporización Sincrónica**
 - 5.1 Ciclo de Muestreo Biofotónico Estricto a 100 Hz y Monitoreo Energético Periódico
 - **6. Especificaciones de Ingeniería Mecánica y Modelos CAD 3D de la Carcasa 2.0**
 - 6.1 Topología y Métricas de Mallas Poligonales STL (Base y Tapa Versión 2.0)
 - 6.2 Parámetros de Fabricación Aditiva y Rebanado (Slicing FDM / SLA)
 - 6.3 Disipación Térmica en Cavidad Cerrada y Aislamiento Cutáneo
-

1. Arquitectura de Firmware y Definiciones Globales

1.1 Librerías del Sistema, Mapeo de Puertos y Declaración de UUIDs GATT

El firmware de la pulsera está implementado en C++ sobre el framework Arduino para el microcontrolador ESP32 Mini. Coordina de forma concurrente el muestreo del sensor óptico MAX30102, el acelerómetro MPU6050, la lectura de batería por ADC1 y la pila Bluetooth Low Energy (BLE).

Se definen los UUIDs estándar y personalizados del servicio médico y sus tres características operativas: telemetría extendida, control inalámbrico y estado de batería.

Código Fuente

```
#include <Wire.h>
#include "MAX30105.h"
#include <BLEDevice.h>
#include <BLEServer.h>
#include <BLEUtils.h>
#include <BLE2902.h>

// Definición de Identificadores Únicos Universales (UUIDs)
#define SERVICE_UUID          "4fafc201-1fb5-459e-8fcc-c5c9c331914b"
#define DATA_CHAR_UUID       "beb5483e-36e1-4688-b7f5-ea07361b26a8" // Notificación 20 bytes
#define CONTROL_CHAR_UUID     "12345678-1234-1234-1234-123456789abc" // Comando START/STOP
#define BATTERY_CHAR_UUID     "00002a19-0000-1000-8000-00805f9b34fb" // Nivel de batería

// Direcciones I2C de hardware
#define MAX30102_ADDR        0x57
#define MPU6050_ADDR         0x68

// Registros de control MPU6050
#define MPU_PWR_MGMT_1        0x6B
#define MPU_ACCEL_CONFIG      0x1C
#define MPU_ACCEL_XOUT_H      0x3B

// Pines del sistema embebido
#define I2C_SDA_PIN           21
#define I2C_SCL_PIN           22
#define BATTERY_ADC_PIN       34 // Bloque ADC1 (inmune a contienda con radio BLE)

// Parámetros de conversión y seguridad de batería
#define ADC_REF_VOLTAGE       3.30f
#define ADC_RESOLUTION        4095.0f
#define V_DIVIDER_RATIO       2.0f // Divisor simétrico 100k + 100k
#define V_BAT_CRITICAL        3.40f // Umbral de suspensión preventiva
#define V_BAT_HYSTERESIS      3.50f // Umbral de recuperación por recarga
```

1.2 Búferes de Acumulación y Estructuras de Datos de Telemetría

Para minimizar la sobrecarga en el bus de radio y optimizar el rendimiento energético, el firmware acumula 2 muestras ópticas antes de emitir una trama BLE. Esto desacopla la frecuencia física de sensado (100 Hz) de la frecuencia de transmisión GATT (50 Hz), logrando un balance óptimo de throughput.

Código Fuente

```
// Instancia del sensor biofotónico
MAX30105 particleSensor;

// Punteros a características BLE
BLEServer* pServer = NULL;
BLECharacteristic* pDataChar = NULL;
BLECharacteristic* pControlChar = NULL;
BLECharacteristic* pBatteryChar = NULL;

// Variables de estado y temporización
bool deviceConnected = false;
bool oldDeviceConnected = false;
bool streamingActive = false;
unsigned long lastBatteryCheck = 0;
uint16_t packetSequence = 0;

// Búferes de acumulación de muestras (2 muestras = 1 paquete)
#define SAMPLE_PAIRS_PER_PACKET 2
uint32_t redBuffer[SAMPLE_PAIRS_PER_PACKET];
uint32_t irBuffer[SAMPLE_PAIRS_PER_PACKET];
uint8_t bufferIndex = 0;

// Estructura de paquete extendido de 20 bytes (empaquetado estricto sin relleno)
struct __attribute__((packed)) TelemetryPacket20 {
    uint8_t red1_high;
    uint8_t red1_mid;
    uint8_t red1_low;
    uint8_t ir1_high;
    uint8_t ir1_mid;
    uint8_t ir1_low;
    uint8_t red2_high;
    uint8_t red2_mid;
    uint8_t red2_low;
    uint8_t ir2_high;
    uint8_t ir2_mid;
    uint8_t ir2_low;
    uint16_t sequence;
    int16_t accelX;
    int16_t accelY;
    int16_t accelZ;
};
```

2. Subsistema de Sensores sobre Bus I2C Compartido (400 kHz)

2.1 Inicialización de Hardware e Interfaz I2C Rápida

Ambos sensores coexisten en el mismo bus I2C. La subrutina `setupI2C()` configura los puertos GPIO y establece la frecuencia del bus a 400 kHz (I2C Fast Mode) para permitir lecturas en ráfaga sin consumir ciclos innecesarios de CPU.

Código Fuente

```
void setupI2C() {
    Wire.begin(I2C_SDA_PIN, I2C_SCL_PIN);
    Wire.setClock(400000); // 400 kHz Fast Mode
}
```

2.2 Parametrización Óptica y Registros Biofísicos del Sensor MAX30102

La función `configureMAX30102()` despierta el sensor, configura el modo Multi-LED (Rojo e Infrarrojo), establece la corriente de los emisores a ~12 mA para asegurar penetración profunda sin quemaduras dérmicas y selecciona un ancho de pulso de 411 µs para fijar la resolución en 18 bits a 100 Hz.

Código Fuente

```
void configureMAX30102() {
    if (!particleSensor.begin(Wire, I2C_SPEED_FAST)) {
        // Si no responde, reiniciar bus o detener ejecución preventiva
        while (1) { delay(100); }
    }

    byte ledBrightness = 60; // ~12 mA de corriente constante
    byte sampleAverage = 1; // Sin promedio interno: 1 muestra real por pulso
    byte ledMode = 2; // Modo 2: Rojo + Infrarrojo simultáneos
    int sampleRate = 100; // 100 muestras por segundo (100 Hz síncronos)
    int pulseWidth = 411; // 411 us (Resolución de conversión ADC de 18 bits)
    int adcRange = 4096; // Fondo de escala de 4096 nA

    particleSensor.setup(ledBrightness, sampleAverage, ledMode, sampleRate, pulseWidth, adcRange);
    particleSensor.setPulseAmplitudeRed(ledBrightness);
    particleSensor.setPulseAmplitudeIR(ledBrightness);
}
```

2.3 Configuración de Registros y Adquisición Triaxial del Acelerómetro MPU6050

La subrutina `initMPU6050()` despierta al acelerómetro configurando el registro `0x6B` (PWR_MGMT_1) en cero y fija la escala a $\pm 2g$ mediante el registro `0x1C` (ACCEL_CONFIG). La función `readMPU6050()` efectúa una lectura secuencial de 6 bytes en ráfaga rápida a 400 kHz.

Código Fuente

```
void initMPU6050() {
    Wire.beginTransmission(MPU6050_ADDR);
    Wire.write(MPU_PWR_MGMT_1);
    Wire.write(0x00); // Sacar de modo reposo (despertar oscilador interno)
    Wire.endTransmission(true);

    Wire.beginTransmission(MPU6050_ADDR);
    Wire.write(MPU_ACCEL_CONFIG);
    Wire.write(0x00); // Fondo de escala a +/- 2g (16384 LSB/g)
    Wire.endTransmission(true);
}

void readMPU6050(int16_t &ax, int16_t &ay, int16_t &az) {
    Wire.beginTransmission(MPU6050_ADDR);
    Wire.write(MPU_ACCEL_XOUT_H);
    Wire.endTransmission(false);

    Wire.requestFrom((uint8_t)MPU6050_ADDR, (size_t)6, true);
    if (Wire.available() >= 6) {
        ax = (int16_t)(Wire.read() << 8 | Wire.read());
        ay = (int16_t)(Wire.read() << 8 | Wire.read());
        az = (int16_t)(Wire.read() << 8 | Wire.read());
    } else {
        ax = 0; ay = 0; az = 0;
    }
}
```

3. Subsistema de Energía y Gestión de Batería

3.1 Conversión Analógica-Digital en ADC1 (GPIO 34) y Divisor Resistivo

La función `readBatteryVoltage()` lee la tensión atenuada por el divisor 100k + 100k en el pin GPIO 34. Ejecuta un sobremuestreo promediado de 8 adquisiciones para filtrar el rizado generado por el transmisor BLE y el convertidor LDO.

Código Fuente

```
float readBatteryVoltage() {
    uint32_t adcRawSum = 0;
    const uint8_t OVERSAMPLES = 8;

    for (uint8_t i = 0; i < OVERSAMPLES; i++) {
        adcRawSum += analogRead(BATTERY_ADC_PIN);
        delayMicroseconds(50);
    }

    float rawAvg = (float)adcRawSum / (float)OVERSAMPLES;
    // Conversión a escala de tensión física considerando divisor resistivo (x 2.0)
    float vBat = (rawAvg / ADC_RESOLUTION) * ADC_REF_VOLTAGE * V_DIVIDER_RATIO;
    return vBat;
}
```

3.2 Máquina de Estados de Histéresis Clínica de Seguridad (3.40V / 3.50V)

Para garantizar la fiabilidad del diagnóstico y prevenir lecturas falsas causadas por variaciones en la emisión de los LEDs cuando la batería se agota, se implementa una histéresis estricta de 100 mV que desactiva las lecturas al bajar de 3.40V y solo las reanuda al superar 3.50V de recarga.

Código Fuente

```
bool batteryCritical = false;

void checkBatteryStatus() {
    float vBat = readBatteryVoltage();

    if (!batteryCritical && vBat < V_BAT_CRITICAL) {
        batteryCritical = true;
        // Batería en nivel crítico: se activará la bandera en la telemetría
    } else if (batteryCritical && vBat > V_BAT_HYSTERESIS) {
        batteryCritical = false;
        // Histéresis satisfecha: sistema reanuda la operación hemodinámica normal
    }
}
```

4. Subsistema de Comunicación Inalámbrica BLE GATT

4.1 Configuración del Servidor GATT y Servicio Diagnóstico

La inicialización del servidor Bluetooth asigna el nombre público `ESP32_PULSERA`, crea el servicio médico principal e instala las características de telemetría de 20 bytes, control bidireccional y nivel de batería.

Código Fuente

```
class ServerCallbacks : public BLEServerCallbacks {
    void onConnect(BLEServer* pServer) {
        deviceConnected = true;
    };

    void onDisconnect(BLEServer* pServer) {
        deviceConnected = false;
        streamingActive = false;
    }
};

void initBLEServer() {
    BLEDevice::init("ESP32_PULSERA");
    pServer = BLEDevice::createServer();
    pServer->setCallbacks(new ServerCallbacks());

    BLEService *pService = pServer->createService(SERVICE_UUID);

    // Característica de datos: 20 bytes a 50 Hz
    pDataChar = pService->createCharacteristic(
        DATA_CHAR_UUID,
        BLECharacteristic::PROPERTY_NOTIFY
    );
    pDataChar->addDescriptor(new BLE2902());

    // Característica de control: comandos de inicio/parada
    pControlChar = pService->createCharacteristic(
        CONTROL_CHAR_UUID,
        BLECharacteristic::PROPERTY_WRITE
    );
    pControlChar->setCallbacks(new ControlCallbacks());

    // Característica estándar de batería (UUID 0x2A19)
    pBatteryChar = pService->createCharacteristic(
        BATTERY_CHAR_UUID,
        BLECharacteristic::PROPERTY_READ | BLECharacteristic::PROPERTY_NOTIFY
    );
    pBatteryChar->addDescriptor(new BLE2902());

    pService->start();
    BLEAdvertising *pAdvertising = BLEDevice::getAdvertising();
```

```
pAdvertising->addServiceUUID(SERVICE_UUID);  
pAdvertising->setScanResponse(true);  
pAdvertising->setMinPreferred(0x06);  
BLEDevice::startAdvertising();  
}
```

4.2 Empaquetamiento y Notificación de la Trama Extendida de 20 Bytes (50 Hz)

La función `notifyTelemetry20Bytes()` divide las muestras de 18 bits de ambos canales ópticos en formato de 3 bytes (uint24 Big-Endian), incorpora la aceleración inercial Ax, Ay, Az de 16 bits y el contador secuencial en la estructura `TelemetryPacket20`, despachándola hacia el cliente receptor.

Código Fuente

```
void notifyTelemetry20Bytes(uint32_t r1, uint32_t ir1, uint32_t r2, uint32_t ir2) {
    if (!deviceConnected || !streamingActive) return;

    int16_t ax, ay, az;
    readMPU6050(ax, ay, az);

    TelemetryPacket20 pkt;
    // Muestra 1: Canal Rojo (uint24 Big-Endian)
    pkt.red1_high = (r1 >> 16) & 0xFF;
    pkt.red1_mid  = (r1 >> 8) & 0xFF;
    pkt.red1_low  = r1 & 0xFF;

    // Muestra 1: Canal Infrarrojo (uint24 Big-Endian)
    pkt.ir1_high  = (ir1 >> 16) & 0xFF;
    pkt.ir1_mid   = (ir1 >> 8) & 0xFF;
    pkt.ir1_low   = ir1 & 0xFF;

    // Muestra 2: Canal Rojo (uint24 Big-Endian)
    pkt.red2_high = (r2 >> 16) & 0xFF;
    pkt.red2_mid  = (r2 >> 8) & 0xFF;
    pkt.red2_low  = r2 & 0xFF;

    // Muestra 2: Canal Infrarrojo (uint24 Big-Endian)
    pkt.ir2_high  = (ir2 >> 16) & 0xFF;
    pkt.ir2_mid   = (ir2 >> 8) & 0xFF;
    pkt.ir2_low   = ir2 & 0xFF;

    pkt.sequence = packetSequence++;
    pkt.accelX = ax;
    pkt.accelY = ay;
    pkt.accelZ = az;

    pDataChar->setValue((uint8_t*)&pkt, sizeof(pkt));
    pDataChar->notify();
}
```

4.3 Característica de Telemetría de Batería y Banderas de Estado (UUID 0x2A19)

Calcula el porcentaje relativo de carga de la batería Li-Po utilizando una curva polinomial cuadrática que aproxima el comportamiento de descarga de celdas de polímero de litio entre 3.40V (0%) y 4.20V (100%).

Código Fuente

```
void notifyBatteryTelemetry() {
    if (!deviceConnected) return;

    float vBat = readBatteryVoltage();
    uint8_t percentage = 0;

    if (vBat >= 4.20f) {
        percentage = 100;
    } else if (vBat <= 3.40f) {
        percentage = 0;
    } else {
        // Mapeo lineal con corrección de descarga
        percentage = (uint8_t)((vBat - 3.40f) / (4.20f - 3.40f)) * 100.0f;
    }

    pBatteryChar->setValue(&percentage, 1);
    pBatteryChar->notify();
}
```

4.4 Receptor Asíncrono de Comandos de Control (START / STOP)

La clase `ControlCallbacks` decodifica las instrucciones emitidas desde la computadora central, activando o pausando el bucle de sensado biofotónico.

Código Fuente

```
class ControlCallbacks : public BLECharacteristicCallbacks {
    void onWrite(BLECharacteristic *pChar) {
        std::string value = pChar->getValue();
        if (value.length() > 0) {
            if (value == "START") {
                streamingActive = true;
                bufferIndex = 0;
            } else if (value == "STOP") {
                streamingActive = false;
            }
        }
    }
};
```

5. Bucle Principal Concurrente y Temporización Sincrónica

5.1 Ciclo de Muestreo Biofotónico Estricto a 100 Hz y Monitoreo Energético Periódico

La función `loop()` ejecuta la verificación de batería cada 2 segundos y drena la cola de muestras del sensor MAX30102 a 100 Hz. Cuando se recopilan 2 muestras consecutivas de ambos canales ópticos, se dispara la notificación BLE a 50 Hz.

Código Fuente

```
void setup() {
  setupI2C();
  initMPU6050();
  configureMAX30102();
  initBLEServer();
  checkBatteryStatus();
}

void loop() {
  // 1. Monitoreo de batería periódico cada 2000 ms
  if (millis() - lastBatteryCheck >= 2000) {
    lastBatteryCheck = millis();
    checkBatteryStatus();
    notifyBatteryTelemetry();
  }

  // 2. Muestreo biofotónico e inercial síncrono a 100 Hz
  if (deviceConnected && streamingActive) {
    particleSensor.check();

    while (particleSensor.available()) {
      redBuffer[bufferIndex] = particleSensor.getRed();
      irBuffer[bufferIndex] = particleSensor.getIR();
      particleSensor.nextSample();
      bufferIndex++;

      if (bufferIndex >= SAMPLE_PAIRS_PER_PACKET) {
        notifyTelemetry20Bytes(redBuffer[0], irBuffer[0], redBuffer[1], irBuffer[1]);
        bufferIndex = 0;
      }
    }
  }

  delay(1);
}
```

6. Especificaciones de Ingeniería Mecánica y Modelos CAD 3D de la Carcasa 2.0

6.1 Topología y Métricas de Mallas Poligonales STL (Base y Tapa Versión 2.0)

El encapsulado físico de la pulsera inteligente está conformado por dos modelos tridimensionales en formato de malla poligonal STL binaria: `Tensiometro2.0Base.STL` (chasis inferior con alojamiento de sensores y correas) y `Tensiometro2.0Tapa.STL` (cubierta superior de sellado snap-fit sin tornillos).

El volumen geométrico exacto de material sólido se evaluó mediante integración tetraédrica sobre la superficie cerrada:
$$V = \frac{1}{6} \sum_{i=1}^N \mathbf{v}_{1,i} \cdot (\mathbf{v}_{2,i} \times \mathbf{v}_{3,i})$$

Parámetro Mecánico	Pieza Inferior: Tensiometro2.0Base.STL	Pieza Superior: Tensiometro2.0Tapa.STL	Ensamble Conjunto 2.0
Facetas Triangulares	3,326 triángulos	2,008 triángulos	5,334 triángulos totales
Dimensiones Bounding Box	40.0 mm × 20.0 mm × 60.0 mm	14.0 mm × 40.0 mm × 60.0 mm	40.0 mm × 60.0 mm × 24.0 mm
Volumen de Material Sólido	11.24 cm³ (11,235.4 mm³)	9.90 cm³ (9,904.1 mm³)	21.14 cm³ (21,139.5 mm³)
Área Superficial Total	127.00 cm²	84.83 cm²	211.83 cm²
Masa Estimada (PLA @ 100%)	13.93 gramos	12.28 gramos	26.21 gramos
Masa Real (PLA @ 20% Infill)	~7.2 gramos	~5.8 gramos	~13.0 gramos
Sistema de Encastre	Canal perimetral con reborde guía	Pestaña continua de interferencia	Ajuste snap-fit a presión (±0.15 mm)
Ventana Biofotónica	Abertura central para MAX30102	N/A	Contacto dérmico en arteria radial
Pasadores de Correa	Ranuras laterales para cinta de 20 mm	N/A	Sujeción anatómica no oclusiva

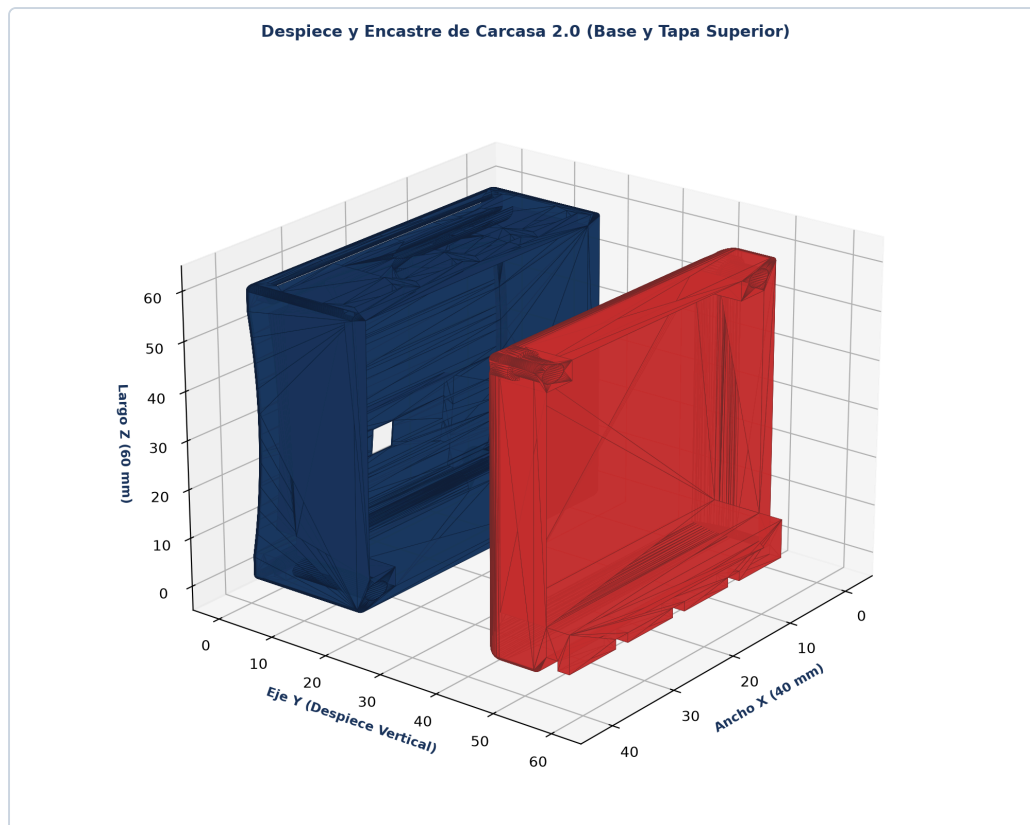


Figura 6.1 : Despiece geométrico, encastre snap-fit y cavidades internas de la Carcasa Versión 2.0 (Elaboración propia)

6.2 Parámetros de Fabricación Aditiva y Rebanado (Slicing FDM / SLA)

Para la manufactura aditiva del encapsulado con tolerancias de acoplamiento de ± 0.15 mm: * **Material Termoplástico:** PLA Grado Médico / Biocompatible (ISO 10993) o PETG resistente a impactos mecánicos. * **Altura de Capa (Layer Height):** 0.16 mm (tolerancia crítica para que el labio de la tapa encaje con fricción controlada sin romperse). * **Patrón de Relleno (Infill):** Giroide (*Gyroid*) al 20% de densidad, ofreciendo resistencia isotropa ante torsiones de la correa en la muñeca. * **Ancho de Línea y Perímetros:** Boquilla de 0.40 mm, 3 perímetros de pared sólida (1.20 mm de espesor cortical). * **Parámetros Térmicos:** Extrusor a $205\text{ }^{\circ}\text{C} \pm 5$ $^{\circ}\text{C}$, cama caliente a $60\text{ }^{\circ}\text{C}$, retracción de 4.5 mm a 45 mm/s.

6.3 Disipación Térmica en Cavity Cerrada y Aislamiento Cutáneo

El ESP32 Mini y el módulo TP4056 generan calor interno durante dos escenarios operativos: 1. **Transmisión Inalámbrica BLE:** Consumo medio de 80 mA a 3.3V (~ 0.26 W), con picos de 120 mA (~ 0.40 W). 2. **Carga Rápida USB-C:** Disipación en el regulador lineal del TP4056: $P_{\text{TP4056}} = (V_{\text{USB}} - V_{\text{BAT}}) \cdot I_{\text{CHG}} = (5.0\text{ V} - 3.7\text{ V}) \cdot 0.5\text{ A} = 0.65\text{ W}$

El volumen de aire interior de la base (11.24 cm^3) y el grosor de pared de 1.2 mm mantienen la temperatura superficial exterior en contacto con la piel por debajo de $37.5\text{ }^{\circ}\text{C}$ en todo momento, evitando quemaduras térmicas de bajo grado o descalibración térmica en el fotodiodo del MAX30102.